

LLM-мониторинг панелей Grafana

Концепция «дежурного аналитика» поверх множества дашбордов

Версия 1.0

10 июля 2026

Технический дизайн-документ

Grafana MCP · Prometheus · Loki · LLM

Документ описывает подход к автоматическому наблюдению за большим числом панелей Grafana с помощью языковой модели. Ключевая идея — не показывать LLM картинки дашбордов, а давать ей числа и метаданные: система сама вытягивает данные из-за панелей, сжимает их в компактную сводку, а модель работает как дежурный инженер — находит аномалии, связывает сигналы между панелями и пишет понятное человеку объяснение.

1 Проблема и подход

Когда дашбордов и панелей десятки или сотни, человек физически не может держать всё в поле зрения. Классические пороговые алерты хорошо ловят «значение превысило порог», но плохо справляются с тремя вещами: корреляцией множества панелей между собой, формулировкой «что это значит» на языке дежурного и приглушением шума от одиночных всплесков.

LLM закрывает именно этот разрыв. Она агрегирует множество слабых сигналов в одну осмысленную картину. Но использовать её нужно аккуратно: скриншоты дороги, медленны и плохо анализируются, а сырые временные ряды из тысяч точек — это лишние токены и деньги. Поэтому модель ставится **вторым уровнем** поверх дешёвых детекторов и получает на вход только сжатые сводки и уже найденные аномалии.

Принцип разделения ролей. MCP — это «руки» (достать метрики и метаданные). Собственный код — «фильтр» (сжать данные и отсеять норму). LLM — «голова» (связать сигналы и объяснить). Модель вызывается только тогда, когда есть что объяснять.

РУКИ

Grafana MCP

Инвентаризация панелей и
запросы к источникам данных

ФИЛЬТР

Свой код

Статистика, базовая линия,
детектор аномалий

ГОЛОВА

LLM

Корреляция, оценка
серьёзности, объяснение

2 Роль Grafana MCP

Официальный сервер [grafana/mcp-grafana](#) предназначен именно для того, чтобы вытягивать данные для анализа, а не просто открывать дашборды. Для нашей задачи важны два уровня доступа.

Метаданные панелей

Поиск дашбордов по названию, получение их деталей и — что особенно ценно — извлечение из каждой панели её заголовка, запроса (PromQL / LogQL) и источника данных. Благодаря этому LLM может сама обойти все дашборды и узнать, какие метрики стоят за панелями, не требуя ручного описания.

Сами метрики

Полноценные запросы к источникам: PromQL к Prometheus в режимах instant и range (то есть и точка, и временной ряд за окно), метаданные метрик, списки имён и лейблов, расчёт перцентилей по гистограммам. Для логов — LogQL к Loki, включая обнаружение паттернов. Дополнительно поддерживаются InfluxDB, CloudWatch, Elasticsearch и другие источники (часть выключена по умолчанию и включается флагами). Полезные бонусы: экспорт панели в PNG, работа с алертами и аннотациями, генерация ссылок-диплинков, а через интеграцию Sift — готовое обнаружение паттернов ошибок и медленных запросов.

Важная оговорка. MCP отдаёт временные ряды «как есть». Поэтому предобработку и сжатие (посчитать статистику, сравнить с базовой линией, отсеять норму) всё равно делает собственный код — до отправки в модель. Иначе на многих панелях анализ выходит дорогим по токенам.

3 Архитектура по слоям

1

Сбор данных

Через Grafana HTTP API / MCP получаем список дашбордов и панелей, а из каждой панели — её запрос к источнику (Prometheus, Loki, ClickHouse, PostgreSQL и т.д.). Дальше либо дёргаем источник напрямую по этому запросу, либо используем Grafana как прокси. На выходе — временные ряды за нужное окно (например, последние 15 минут плюс та же неделя назад для сравнения).

2

Предобработка — ключевой шаг экономии

LLM нельзя кормить тысячи точек. До модели детерминированным кодом считаем статистику: текущее значение, среднее, перцентили, тренд, отклонение от базовой линии, скорость роста, сравнение «сейчас против часа / дня / недели назад». Часто здесь же работает классический детектор аномалий (z-score, EWMA, Prophet / ADTK), а модель получает только подозрительные панели, а не все подряд.

3

Анализ LLM

Модели отдаём компактный пакет: имя панели, её смысл, пороги / SLO, сжатую статистику и список сработавших аномалий по нескольким панелям сразу. Задача модели — то, что плохо формализуется правилами: оценить серьёзность, связать сигналы между панелями, отсеять ложные срабатывания и написать короткое объяснение с гипотезой причины.

4

Доставка

Результат уходит в Slack / Telegram / почту. Срочные инциденты — сразу, с ссылкой на дашборд и при желании PNG панели. Несрочное копится в дневной дайджест: «что в целом происходило и на что обратить внимание». Дополнительно можно вести живой журнал состояния как обновляемый артефакт.

5

Расписание

Периодический запуск по cron: частый лёгкий проход по критичным панелям плюс редкий глубокий обзор всего парка дашбордов.

4 Сценарий работы

Фаза 0 — Разовая инвентаризация

Раз в сутки или при изменении дашбордов.

LLM через MCP проходит по всем дашбордам и вытаскивает из панелей заголовки, запрос и источник. Получается «карта наблюдения»:

panel_id	dashboard	title	datasource	query
p-142	API	p99 latency	prom-main	histogram_quantile(0.99, ...)
p-143	API	5xx rate	prom-main	sum(rate(http_5xx[5m]))
p-201	DB	connections used	prom-main	pg_stat_activity_count

Реестр вы храните у себя (файл или БД) и один раз обогащаете смыслом: для каждой панели добавляете слой (api / db / infra), порог или SLO и критичность. Именно этот контекст делает выводы модели осмысленными.

Фаза 1 — Периодический сбор

Каждые 1–5 минут. LLM ещё не участвует.

По расписанию собственный код идёт по реестру и для каждой критичной панели дёргает через MCP запрос к Prometheus в режиме range за два окна: «сейчас» (последние 15 минут) и «база» (то же время неделю назад). Для логов — запрос к Loki или детектор паттернов. Это дешево и детерминированно.

Фаза 2 — Предобработка и фильтр

Собственный код. LLM вызывается, только если что-то найдено.

Из сырых рядов считается сводка и прогоняется дешёвый детектор (z-score / EWMA). На выходе — только подозрительные панели плюс их соседи по слою:

```
[
  {"panel": "API / p99 latency", "now": 820, "baseline": 210, "unit": "ms",
    "delta": "+290%", "trend": "растёт 8 мин", "slo": "<300ms", "status": "BREACH"},
  {"panel": "API / 5xx rate", "now": 47, "baseline": 0.3, "unit": "req/s",
    "delta": "x150", "trend": "скачок 6 мин назад", "status": "BREACH"},
  {"panel": "DB / connections used", "now": 198, "baseline": 60, "max": 200,
    "delta": "почти лимит", "status": "WARN"}
]
```

Если аномалий нет — модель не вызывается вовсе. Экономятся и токены, и деньги.

Фаза 3 — LLM как «дежурный аналитик»

Только теперь пакет уходит в модель. Примерный промпт:

```
# SYSTEM / PROMPT
Ты — SRE-дежурный. Ниже сводка метрик Grafana за последние 15 минут
с базовой линией (та же метрика неделю назад).
Числа НЕ выдумывай — опирайся только на переданные.

Задача:
1. Определи, есть ли реальный инцидент или это шум.
2. Свяжи сигналы между панелями в одну гипотезу причины.
3. Оцени серьёзность: P1 / P2 / P3.
4. Дай 1-2 предложения «что проверить первым».

Формат ответа:
severity | суть одним предложением | гипотеза причины | первый шаг
Ссылайся на конкретные значения из данных.

ДАННЫЕ:
<json из фазы 2>
```

Ожидаемый ответ модели:

```
P1 | p99 latency вырос до 820ms (+290%) одновременно со всплеском
5xx (x150) и подходом пула БД к лимиту (198/200).
Гипотеза: исчерпание пула коннектов к БД → таймауты → рост latency и 5xx.
Первый шаг: проверить долгие/зависшие запросы в БД и недавние деплои API.
```

Модель сделала то, что пороги не умеют: **склеила три панели в один диагноз** и написала это на языке дежурного.

Фаза 4 — Доставка

Инциденты P1 / P2 уходят сразу в Slack / Telegram: текст от модели плюс ссылка-диплинк на дашборд (через MCP) и, при желании, PNG панели. Уровень P3 и «всё спокойно» копят в дневной дайджест: «за сутки 2 инцидента, оба вокруг пула БД — стоит поднять лимит».

5 Расписание

Разумная раскладка запусков по cron (через scheduled tasks):

Задача	Частота	Что делает	Вызов LLM
Инвентаризация панелей	Раз в сутки	Обновляет карту наблюдения из дашбордов	Опционально
Лёгкий проход	Каждые 1–5 мин	Сбор метрик по критичным панелям + фильтр	Только при аномалии
Дневной дайджест	Раз в день утром	Сводка инцидентов и трендов за сутки	Да

6 Риски и как их закрывать

Риск	Как закрываем
Стоимость и расход токенов	Предобработка: модель видит только сжатые сводки и найденные аномалии, а не сырые ряды.
Галлюцинации и выдуманные числа	Модель не считает, а интерпретирует переданные значения; в промпте требуем ссылок на конкретные числа.
Ложные срабатывания и шум	LLM — второй уровень поверх дешёвых детекторов, а не единственный триггер; фильтр отсекает норму до модели.
Пропуск инцидента	Соседи по слою всегда идут в пакет вместе с аномальной панелью — это даёт контекст для корреляции.

7 Итог

Схема сводится к одному принципу: **MCP достаёт данные, собственный код их сжимает и фильтрует, а LLM связывает сигналы и объясняет — и вызывается только тогда, когда есть что объяснить.** Такой подход даёт наблюдение за десятками панелей без постоянных расходов на модель, с понятными человеку выводами и минимумом шума.

Следующий практический шаг — рабочий скелет: Python-скрипт, который ходит в Grafana MCP, считает сводку и вызывает LLM, плюс готовая scheduled task под выбранное расписание. Для него потребуется зафиксировать источники (Prometheus / Loki) и канал алертов (Slack / Telegram).